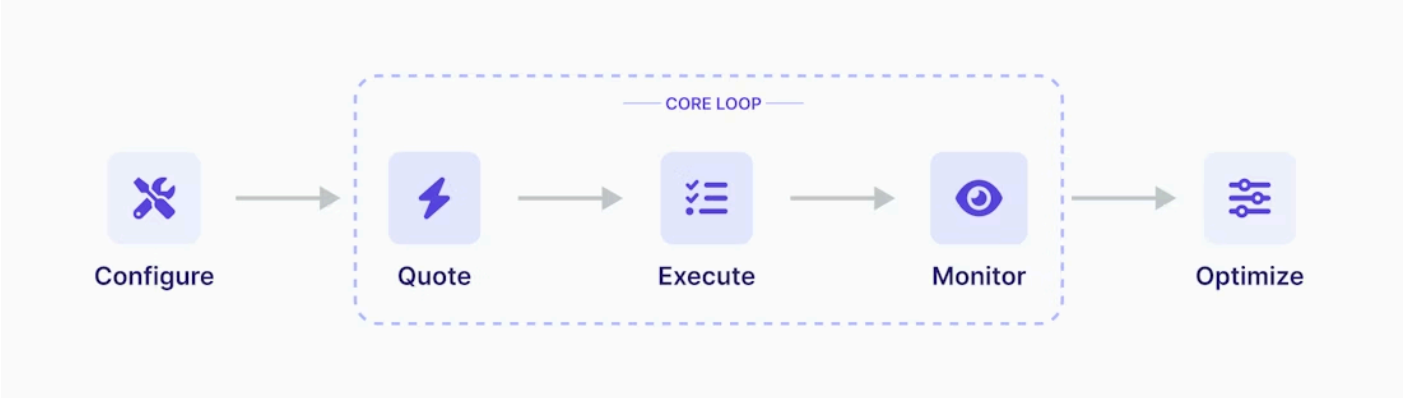
Quickstart

Copy page

Execute your first cross-chain transaction in under 5 minutes.

Relay is a multichain payments network. You define the **Intent** (what the user wants), and Relay handles the **Execution** (how to get there).

Follow this 5-step flow to integrate Relay into your application:



1 Configure

To see how fast and simple Relay makes transacting cross-chain, let's bridge real assets between inexpensive L2s—Base and Arbitrum.

Prerequisites:

- **Base URL:** `https://api.relay.link`
- **Environment:** Node.js installed
- **Wallet:** An EOA with a small amount of ETH (~$2.00 USD equivalent) on Base

API Key Provisioning

You can find a self-serve form to request API keys **here**.
Having an API key will allow you to use the API at a higher rate limit.

Chain Configuration

For chain metadata we recommend querying the **chains API** in your application. This will return some valuable information about the chain, we've provided a sample below:

Response Request

```
{
  "chains": [{
      "id":10,
      "name":"optimism",
      "displayName":"Optimism",
      "httpRpcUrl":"https://optimism.publicnode.com",
      "wsRpcUrl":"wss://optimism.publicnode.com",
```

··· See all 87 lines

The endpoint exposes a vast amount of chain metadata but the important things to note are:

- `vmType` : This is the type of virtual machine that the chain uses (SVM, EVM, BVM, etc).
- `id` : This is the chain id of the chain. Some chains like Bitcoin/Solana/Tron have a custom id that the Relay team chose upon deployment.
- `disabled` : This is a boolean that indicates if the chain is disabled. When incidents arise we may disable a chain to prevent users from interacting with it.
- `blockProductionLagging` : This boolean indicates if the chain is lagging in block production. You can use this to let your users know about degraded Relay performance in real time.

2 Quote

Every action in Relay starts with a Quote.

The **quote endpoint** handles all of your use cases, whether it's a **bridge**, **swap**, or **cross-chain call**. It calculates fees, finds the best route, and generates the transaction data.

Request

As an example, let's consider the scenario of bridging 0.0001 ETH from Base (Chain ID 8453) to Arbitrum One (Chain ID 42161):
Note: the `requestId` *will be unique to every request.*

Request Response

```
curl -X POST "https://api.relay.link/quote/v2" \
-H "Content-Type: application/json" \
-d '{
  "user": "YOUR_WALLET_ADDRESS",
  "originChainId": 8453,
  "destinationChainId": 42161,
  "originCurrency": "0x0000000000000000000000000000000000000000",
  "destinationCurrency": "0x0000000000000000000000000000000000000000",
  "amount": "100000000000000",
  "tradeType": "EXACT_INPUT"
}'
```

3 Execute

The **quote endpoint** returns a `steps` array. Think of this as a recipe your application must follow. You need to iterate through these steps and prompt the user to sign or submit them.

RELAY Deep Dive

For the full logic on parsing steps, see **Understanding Step Execution**.

The Logic

For a simple ETH bridge, the `steps` array contains a single transaction item. To execute it:

- **Check the Step Kind:** Identify if the step is a `transaction` (submit to chain) or a `signature` (sign off-chain).
- **Execute:** For transactions, submit the provided `data`, `to`, `value`, and `chainId` using the user's wallet. For signatures, follow the `signatureKind` provided in the step item to sign the `message`.

The Script (Node.js / Viem example)

Copy the script below to execute the transaction returned by the Quote in Step 2. You'll need to add the wallet connection logic using your preferred provider.

```
// npm install viem
import { createWalletClient, http } from 'viem';
import { privateKeyToAccount } from 'viem/accounts';
import { base } from 'viem/chains';

// 1. Setup Wallet (Base)
//Initialize your wallet using a provider or your preferred method
```

··· See all 46 lines

4 **Monitor**

Once you submit the transaction, the Relay Solver detects the deposit and fills the request on the destination chain.

Use the **status endpoint** with the `requestId` located inside each step object in your **quote response** to track status and confirm success. You can also use the `check.endpoint` property inside the step item object as the endpoint for checking the status of the request.

Note: Replace with the requestId returned in your quote

Request Response

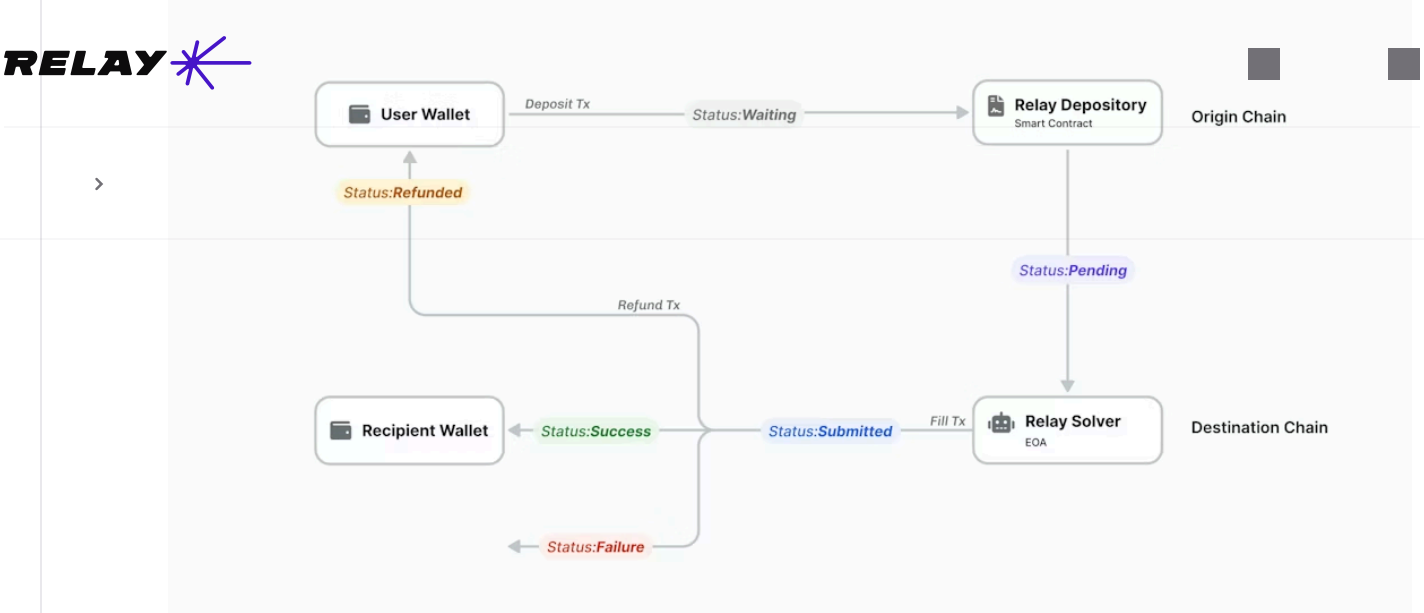
```
curl "https://api.relay.link/intents/status/v3?requestId=0x20538510fd9eab7a
```

Poll this endpoint once per second. For a real-time status stream you can subscribe to Relay's .

Status Lifecycle

The diagram below illustrates the journey of a cross-chain transaction through Relay's infrastructure and the corresponding API statuses. Below are descriptions for each status. Refer to the endpoint for a full list of statuses and what they mean.

- `waiting` : The user is submitting or has submitted the deposit transaction to the Relay Depository contract on the origin chain, but it has not yet been indexed by the system.
- `pending` : The deposit was successfully indexed. The Relay Solver is now monitoring the request and preparing the fill transaction on the destination chain.
- `success` : The Relay Solver successfully executed the "Fill Tx", and the funds have reached the recipient.



The diagram also depicts paths where a transaction may end up in a failure or refund state if the fill cannot be completed. Learn more about how Relay helps you handle **refunds**.

5 **Optimize**

You have successfully executed your first cross-chain transaction with Relay! Check out some of the advanced features we offer to customize the experience:

- **App Fees**: Monetize your integration by adding a fee (in bps) to every quote. Revenue is collected automatically in USDC.
- **Smart Accounts**: Use **ERC-4337** and **EIP-7702** to enable *Gas Sponsorship* and *Atomic Batching* (e.g., Approve + Swap in one click).
- **Transaction Indexing**: Handle complex settlement scenarios, like tracking complex same-chain wraps or transfers.

Try it in a Sandbox

Experiment with the Quickstart code in our **interactive sandbox**. You can fork or clone it to build your own project with Relay.

See Also

- **Bridging & Onboarding**: Learn more about instant cross-chain deposits and withdrawals.
- **Swaps**: Combine bridging with DEX meta-aggregation for any-to-any token swaps.
- **Call Execution**: Execute arbitrary transactions on any chain, paying with any token.

Was this page helpful? Yes No

Powered by mintlify